```
$ordered = [
    'a' => $a,
    'b' => $b,
    'c' => $c,
];

asort($ordered);

foreach ($ordered as $k => $arr) {
    print "\${$k} size is " . sizeof($arr) . "\n"; // Output will vary
                                                   because of the
                                                   random sizes
}

Output:
$c size is 1
$b size is 4
$a size is 8
```

Here we first create three randomly sized arrays using the built-in `array_pad()` function. Next, we store them in a parent array `$ordered`. Finally, we pass this array of arrays to `asort()`. `array_pad()` is a built-in function that adds elements to the end (or beginning) of an array, the value of each added element is specified in the third argument. Thus, `array_pad([0], rand(0, 10), 0)` simply means "add a random amount (between zero and ten) of zeroes to the input array". The effect is that `$a`, `$b`, and `$c` are now of random lengths (this example only cares about array size, not the values of the elements). When PHP sorts arrays, it compares the number of elements, and our output will show us that the arrays in `$ordered` are now in ascending size order.

If the array contains objects, then it is sorted according to object properties. The sorting rules for object arrays are complex; in fact, they're downright arcane and take into account the number of public and private properties, then the values of those properties, then the class name of the object. The code examples to demonstrate `sort()` behavior would take up multiple pages and leave most readers (and probably the author) none the wiser; there's little value in doing so. When it comes to sorting arrays of objects, or arrays of mixed types, it's far better to define your own rules, which you can do as follows…